CSED233 · 데이터구조 · 포스텍

정리본 4. Lists, Stacks, Queues

선형 자료구조 3종: List ADT와 구현 방식, Stack(LIFO), Queue(FIFO), Deque.

1. List(리스트) — 순서가 있는 컬렉션

List(리스트): 순서가 있는 유한 개의 원소 모음. L = ⟨ a_1, a_2, ⋯, a_n ⟩. 중복 허용, 공 리스트(n=0) 가능.



Fence(|)는 List를 왼쪽 구간과 오른쪽 구간으로 나누는 현재 위치 표시이다. 왼쪽 구간을 처리 완료, 오른쪽 구간을 미처리 구간으로 해석할 수도 있다. 초기 상태: L = ⟨ 20, 23 | 12, 15 ⟩. Insert(10) 실행 결과: L = ⟨ 20, 23 | 10, 12, 15 ⟩ — Insert는 Fence 바로 오른쪽에 원소를 삽입하며 Fence의 논리적 위치를 유지한다. 이후 Next()를 실행하면 Fence가 오른쪽으로 한 칸 이동하여 ⟨ 20, 23, 10 | 12, 15 ⟩가 된다.

Sorted vs Ordered

- **Sorted List(정렬 리스트)**: 원소가 비교 기준에 따라 정렬된 상태를 유지하는 List. 예: ⟨ 85, 88, 90, 92, 95 ⟩. 새로운 원소를 정렬 조건에 맞는 위치에 삽입하면 전체를 다시 정렬하지 않아도 된다.
- **Ordered List(순서가 있는 리스트)**: 각 원소에 첫 번째, 두 번째 등 논리적 위치가 존재하는 List. 원소의 위치순은 값의 크기순이나 삽입 시간순일 필요가 없다. 예: ⟨ 90, 85, 95, 88, 92 ⟩.
- 핵심 구분: ordered는 원소 사이에 위치 관계가 있음을 뜻하고, sorted는 값이 특정 비교 기준을 만족하도록 배열되었음을 뜻한다. "순서가 있다(ordered)"는 말이 "정렬됐다(sorted)"는 뜻이 아니다.

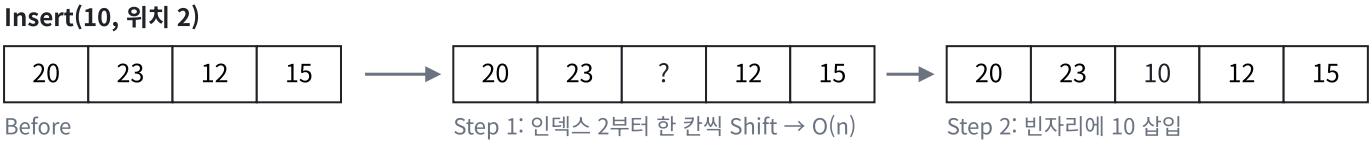
1.1 List의 기본 연산 8가지

연산	설명	시간복잡도
Insert(x, p, L)	위치 p에 x 삽입 (p는 인덱스 또는 노드 참조)	배열 O(n)(끝 삽입은 여유 공간 시 O(1)) · 단일 연결: 숫자 인덱스면 O(n), 직전 노드 참조가 있으면 O(1)
Delete(p, L)	위치 p의 원소 삭제	배열 O(n)(마지막 원소 삭제는 O(1)) · 단일 연결: 숫자 인덱스면 O(n), 직전 노드 참조가 있으면 O(1)
Search(x, L)	원소 x 찾기	배열 O(n) · 단일 연결 O(n) (일반적인 선형 탐색)
Locate(x, L)	x의 인덱스 반환	배열 O(n) · 단일 연결 O(n) (일반적인 선형 탐색)
Retrieve(p, L)	인덱스 p의 값 반환	배열 O(1) · 단일 연결은 숫자 인덱스 p로 접근하면 O(n)
Next(p, L)	p의 다음 위치	배열은 숫자 인덱스 증가 O(1) · 단일 연결은 현재 노드 참조로 다음 노드 이동 O(1)
Previous(p, L)	p의 이전 위치	배열은 숫자 인덱스 감소 O(1) · 단일 연결은 현재 노드 참조에서 이전 노드 탐색 O(n) (이중 연결이면 O(1))
MakeNull(L)	L을 공 리스트로	배열은 원소 수만 초기화하면 O(1) · 모든 노드를 명시적으로 해제하면 O(n)

주의: 숫자 인덱스 p의 다음·이전 숫자를 계산하는 것은 O(1)이지만, 해당 인덱스의 실제 연결 리스트 노드를 찾는 것은 O(n)일 수 있다.

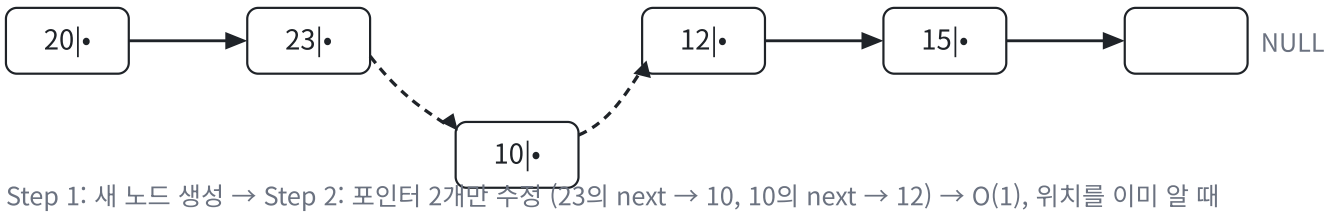
2. 구현: 배열 기반 vs 포인터 기반

2.1 배열 기반 — 연속 메모리



배열 삽입: Shift 비용 때문에 O(n). 맨 뒤 삽입만 O(1) (끝에 여유 공간이 있을 때; 재할당 시 O(n), 분할상환 O(1)).

2.2 포인터 기반 — 연결 리스트



Step 1: 새 노드 생성 → Step 2: 포인터 2개만 수정 (23의 next → 10, 10의 next → 12) → O(1), 위치를 이미 알 때

연결 리스트 삽입: 데이터 이동 없이 포인터 수정만

> **Dummy Header(더미 헤더)**
>
> 연결 리스트 맨 앞에 데이터가 없는 빈 노드를 두는 기법. Dummy Header를 사용하면 첫 번째 실제 노드의 삽입·삭제에도 항상 직전 노드인 dummy가 존재하므로 헤드 포인터 변경을 위한 특수 분기를 줄이고 포인터 수정 코드를 통일할 수 있다.
>
> 구조: `L → [dummy] → a → b → c` . 다만 빈 리스트의 삭제, 유효하지 않은 위치, 마지막 노드 삭제 후 Tail 갱신 등의 경계 조건은 여전히 별도로 처리해야 한다.

2.3 Cursor(커서) 기반 — 배열로 포인터 흉내

배열 인덱스를 포인터처럼 쓰는 기법. 인덱스 0은 NULL 역할로 예약한다.

Index	Data	Next	비고
0	—	—	NULL 역할 (예약)
1	'A'	4	L1의 첫 노드
2	'X'	6	L2의 첫 노드
3	—	5	Free(빈 공간) 리스트 시작
4	'B'	7	L1의 둘째 노드
5	—	0	Free 리스트 끝
6	'Y'	0	L2의 끝
7	'C'	0	L1의 끝

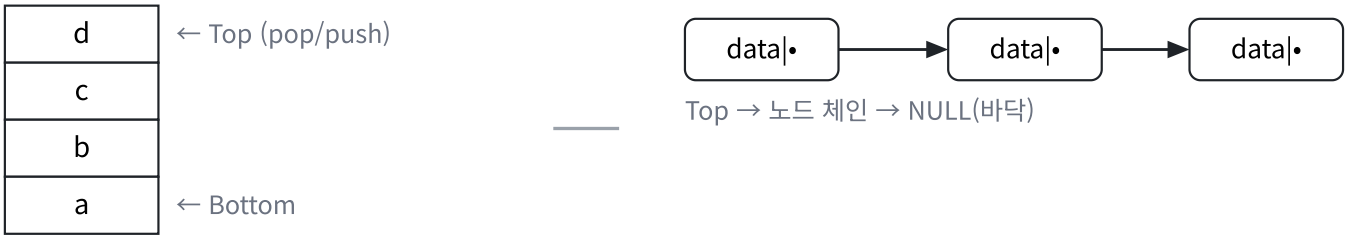
읽는 법: L1 = 1(A) → 4(B) → 7(C, next 0 = 끝). L2 = 2(X) → 6(Y). Free = 3 → 5.

2.4 성능 비교

연산	배열 (Array)	포인터(링크드)	선택 기준
Insert / Delete	O(n)	직전 노드 참조가 있으면 O(1), 위치 탐색부터 하면 O(n)	삽입·삭제가 잦고 직전 노드 참조를 확보할 수 있으면 링크드
Search / Locate	O(n)	O(n)	동등
Random Access	O(1)	O(n)	접근 많으면 배열
메모리	연속	분산	캐시 효율은 배열
구현	간단	복잡	학습 곡선은 배열

3. Stack(스택) — LIFO

Stack(스택): Last In First Out — 마지막에 들어온 것이 먼저 나간다. 현실의 접시 쌓기.



push(a) → push(b) → push(c) → push(d) 순으로 넣으면 pop()은 d부터

- push(x), pop(), top(), isEmpty() — 고정 용량 배열 Stack 또는 Top 포인터를 사용하는 연결 Stack에서는 모든 연산 O(1). 동적 배열 Stack의 push는 분할상환 O(1), 용량 확장이 발생하는 단일 연산의 최악 시간은 O(n)이다.

- 응용: 함수 호출 스택(Call Stack — main → funcA → funcB, 종료는 역순), 괄호 검증(`([{}])` 올바름 / `([}])` 오류), Undo.

예제 · Multiple Stacks — 배열 하나에 스택 둘

1차원 배열의 양 끝에서 마주 보며 자라게 하면 공간을 공유할 수 있다.

```
[ Stack1 →  ...빈 공간...  ← Stack2 ]
 Top1 →                ← Top2
 [0]                   [n-1]
```

Overflow 조건: Top1 + 1 == Top2 (두 탑이 맞닿는 순간). 한쪽이 적게 쓰면 다른 쪽이 남은 공간을 쓸 수 있어 메모리 효율적.

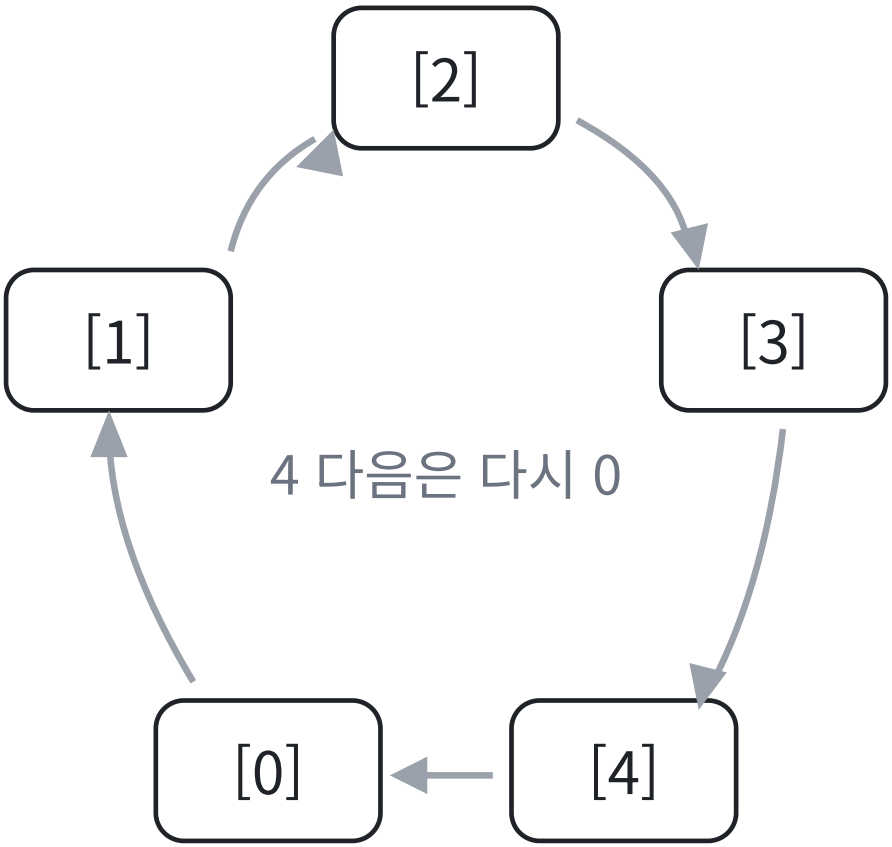
4. Queue(큐) — FIFO

> **Queue(큐)**: First In First Out — 먼저 들어온 것이 먼저 나간다. 현실의 은행 번호표.

- `enqueue(x)` : Rear(뒤)에 삽입 · `dequeue()` : Front(앞)에서 제거·반환 · Front와 Rear를 관리하는 원형 배열 Queue 또는 두 포인터를 유지하는 연결 Queue에서는 모두 O(1). 매 삭제 시 배열을 Shift하는 단순 구현은 dequeue가 O(n)이다.
- 응용: 프린트 대기열, 콜센터, BFS.

4.1 Circular Queue(원형 큐)

일반 배열 큐는 enqueue/dequeue를 반복하면 Front와 Rear가 계속 오른쪽으로 이동해 **앞에 빈칸이 있어도 Rear가 끝에 닿으면 삽입 불가**가 된다. 해결책은 배열 끝과 처음을 논리적으로 연결하는 것 — Modulo(나머지) 연산으로 인덱스를 돌린다.



원형 큐: 인덱스가 끝나면 처음으로

인덱스 갱신 공식

$$\text{Rear} \leftarrow (\text{Rear} + 1) \bmod size$$

$$\text{Front} \leftarrow (\text{Front} + 1) \bmod size$$

Empty와 Full의 구분

배열 크기를 size라 하고, Front는 다음으로 꺼낼 위치, Rear는 다음으로 삽입할 위치를 가리킨다고 하자. 배열의 모든 칸을 사용하면서 Front와 Rear의 위치만으로 상태를 표현하면, 비어 있을 때와 모두 채워졌을 때 **Front == Rear**가 되어 구분할 수 없다. 해결책(택 1):

1. **한 칸 비워두기** (가장 흔함): Empty는 Front == Rear, Full은 아래 조건으로 판정하고, 실제 최대 저장 개수는 size − 1이다.

$$(\text{Rear}+1) \bmod size = \text{Front} \;\Rightarrow\; \text{Full}$$

2. **count 변수**로 원소 개수를 별도 추적: Empty는 count == 0, Full은 count == size, 최대 저장 개수는 size이며 enqueue 시 count를 1 증가, dequeue 시 1 감소시킨다.

4.2 Linked Queue(연결 큐)

포인터로 노드를 체인처럼 연결: Front 포인터 → [Data|Next] → ⋯ → [Data|NULL] ← Rear 포인터. 고정 용량 배열 Queue는 최대 크기를 미리 정해야 하며 용량 초과 시 Overflow가 발생한다. 반면 연결 Queue는 필요한 만큼 노드를 동적으로 할당하여 확장할 수 있어 원소 수가 크게 변하거나 상한을 알기 어려울 때 유리할 수 있다. 다만 동적 배열 기반 Queue도 재할당을 통해 크기를 확장할 수 있다.

5. Deque(덱) — Double-Ended Queue

양쪽 끝에서 삽입/삭제가 모두 가능한 큐. Stack + Queue의 기능을 합친 인터페이스로 보면 된다.

- 이중 연결 리스트 또는 고정 용량 원형 배열 구현에서는 양쪽 끝 삽입·삭제·원소 확인이 모두 O(1)이다. 동적 원형 배열의 용량 확장이 발생하면 해당 삽입의 최악 시간은 O(n), 분할상환 시간은 O(1)이다 (임의 위치 접근은 구현에 따라 다름).
- 응용: 슬라이딩 윈도우 등 앞뒤 접근이 모두 필요한 경우.

6. 선택 가이드

상황 → 자료구조

- 최근 것부터 처리 → **Stack** (Undo, 함수 호출, 뒤로가기)
- 먼저 온 것부터 처리 → **Queue** (프린트 대기열, 콜센터)
- 양쪽에서 접근 → **Deque**
- 삽입·삭제가 잦고 해당 위치의 노드 또는 직전 노드 참조를 이미 확보할 수 있다면 → **Linked List**. 반복적인 인덱스 접근·위치 검색·캐시 효율이 중요하다면 → **Array**
- 인덱스로 빠르게 접근 → **Array**

7. Quiz

Q1. Stack과 Queue의 가장 큰 차이는?

힌트

"어느 쪽에서 나가는가"가 다르다: 스택은 넣은 쪽, 큐는 반대쪽.

정답

개념은 힌트 참고.

정답: Stack은 LIFO(마지막에 들어온 것 먼저), Queue는 FIFO(먼저 들어온 것 먼저).

Q2. 배열 기반 List에서 맨 앞에 삽입하는 비용은?

힌트

섹션 2.1의 Shift 그림: 앞에 넣으면 모든 원소가 움직인다.

정답

개념은 힌트 참고.

① 맨 앞 삽입은 n개 전부를 오른쪽으로 한 칸씩 Shift해야 한다.

② 따라서 O(n). 맨 뒤 삽입만 Shift가 없어 O(1).

정답: O(n) — 모든 원소를 Shift해야 한다. 맨 뒤 삽입만 O(1).

Q3. 연결 리스트에서 Insert는 O(1)인데 Locate는 O(n)인 이유는?

힌트

O(1) Insert에는 "위치를 이미 안다"는 전제가 붙는다.

정답

개념은 힌트 참고.

① Insert: 위치를 알면 포인터 2개만 수정 → O(1).

② Locate: 원소가 어디 있는지 모르니 처음부터 순회 → O(n).

정답: Insert는 위치를 알 때 포인터만 바꾸면 되지만, Locate는 찾으려면 순회가 필요하기 때문.

Q4. Stack을 배열로 구현할 때의 장점과 단점은?

힌트

배열의 일반적 성질(연속 메모리, 고정 크기)을 스택에 대입해 보자.

정답

개념은 힌트 참고.

정답: 장점 — 구조가 간단하고 Top 원소 접근 및 일반적인 push/pop 연산이 O(1)이다. 단점 — 고정 용량 배열을 사용하면 크기 제한에 따른 Overflow 또는 사용하지 않는 공간의 낭비가 발생할 수 있다. 동적 배열 Stack은 용량 확장이 가능하지만 확장 시 재할당과 원소 복사에 O(n)이 소요될 수 있다.

Q5. 왜 Deque을 따로 배우는가?

힌트

Stack도 Queue도 한쪽 끝만 빠르다. 양쪽이 다 필요하면?

정답

개념은 힌트 참고.

정답: 양쪽 끝에서의 빠른 접근이 동시에 필요한 경우(슬라이딩 윈도우 등)가 있어서. Stack+Queue 기능을 하나로 제공한다.

Q6. Singly Linked List를 Doubly Linked List로 바꾸려면?

힌트

뒤로 가려면 뒤를 가리키는 포인터가 필요하다. 비용도 같이 생각해 보자.

정답

개념은 힌트 참고.

① Node 구조체에 `Node* prev` 필드를 추가한다.

② Insert/Delete 시 `prev` 포인터도 함께 갱신한다.

③ 대가: 노드당 포인터 1개분 공간 추가. 얻는 것: 양방향 탐색.

정답: `prev` 포인터를 추가하고 갱신 로직을 보강한다. 공간은 늘지만 양방향 탐색을 얻는다.

Q7. Circular Queue에서 Front == Rear일 때 Empty/Full을 구분 못하는 근본 이유는?

힌트

상태의 개수를 세어 보자. 크기 n 배열의 "채워진 정도"는 0개~n개로 n+1가지다.

정답

개념은 힌트 참고.

① 크기 n 배열의 가능한 채움 상태는 n+1가지(0개~n개).

② 그런데 Front == Rear 조건만으로는 Empty와 Full이 같은 표현으로 보인다.

③ 그래서 한 칸을 희생하거나 count 변수로 상태 공간을 분리한다.

정답: F == R 표현 하나에 Empty와 Full 두 상태가 겹치기 때문 → count 변수나 한 칸 희생으로 구분한다.

Q8. Cursor List에서 link = 0을 끝(sentinel)으로 쓰는 이유는?

힌트

포인터 세계의 NULL을 인덱스 세계에서 무엇으로 대신할까?

정답

개념은 힌트 참고.

정답: 인덱스 0을 "유효 노드 없음"으로 예약해 NULL을 대체한다. 실제 노드는 인덱스 1부터 시작.